

how-to-regras.md

Todos assumem que:

```
from ryu.ofproto import ofproto_v1_3
```

e funções auxiliares típicas:

```
def add_flow(self, datapath, priority, match, actions, buffer_id=None):  
    ...
```

e que as portas podem ser identificadas como segue:

- Subir Mininet com mesh4.
- Rodar **net** e **links** para enxergar as ligações.
- Rodar **sh ovs-ofctl show sX -O OpenFlow13** para cada switch.
- Montar tabelinha porta ↔ vizinho.
- Codificar constantes no controlador.
- Usar essas constantes nas regras SDN toda vez que precisar das portas.

1) Regra SDN para identificar tráfego QUIC-sim

QUIC-sim usa UDP/4433. Logo, o **match** é:

```
match = parser.OFPMatch(  
    eth_type=0x0800, # IPv4  
    ip_proto=17,      # UDP  
    udp_dst=4433  
)
```

Esse match é usado em *todas* as regras abaixo.

2) Forçar rota superior (s1 → s2 → s4)

Dentro do evento `packet_in`, após identificar qual switch (via `dpid`), defina:

Exemplo no switch s1

```
if dpid == 1:
    match = parser.OFPMatch(eth_type=0x0800, ip_proto=17, udp_dst=4433)
    actions = [parser.OFPActionOutput(PORTO_PARA_S2)]
    self.add_flow(datapath, priority=200, match=match, actions=actions)
```

No switch s2

```
if dpid == 2:
    match = parser.OFPMatch(eth_type=0x0800, ip_proto=17, udp_dst=4433)
    actions = [parser.OFPActionOutput(PORTO_PARA_S4)]
    self.add_flow(datapath, 200, match, actions)
```

No switch s4 (último hop)

```
if dpid == 4:
    match = parser.OFPMatch(eth_type=0x0800, ip_proto=17, udp_dst=4433)
    actions = [parser.OFPActionOutput(PORTO_PARA_H2)]
    self.add_flow(datapath, 200, match, actions)
```

3) Forçar rota inferior (s1 → s3 → s4)

No s1:

```
if dpid == 1:
```

```
match = parser.OFPMatch(eth_type=0x0800, ip_proto=17, udp_dst=4433)
actions = [parser.OFPActionOutput(PORTO_PARA_S3)]
self.add_flow(datapath, 200, match, actions)
```

No s3:

```
if dpid == 3:
    match = parser.OFPMatch(eth_type=0x0800, ip_proto=17, udp_dst=4433)
    actions = [parser.OFPActionOutput(PORTO_PARA_S4)]
    self.add_flow(datapath, 200, match, actions)
```

4) ECMP-sim (Balanceamento de Caminhos)

Simple alternância aleatória na porta de saída:

```
import random

if dpid == 1:
    match = parser.OFPMatch(eth_type=0x0800, ip_proto=17, udp_dst=4433)
    out_port = random.choice([PORTO_PARA_S2, PORTO_PARA_S3])
    actions = [parser.OFPActionOutput(out_port)]
    self.add_flow(datapath, 200, match, actions)
```

Essa implementação provoca jitter e comportamento instável — ideal para análise no trabalho.

5) Bloquear QUIC-sim

Simplemente **não definir ações**.

```
if dpid == 1:
    match = parser.OFPMatch(eth_type=0x0800, ip_proto=17, udp_dst=4433)
    actions = [] # sem ações → DROP
```

```
self.add_flow(datapath, 500, match, actions)
```

Use prioridade alta (p.ex. 500) para garantir que override outros caminhos.

6) Bloquear handshake QUIC-sim

O handshake possui "seq": 0 e "type": "handshake" no protocolo. Mas o switch só vê camadas L2/L3/L4, então **não consegue ver o conteúdo**.

O que podemos bloquear é:

- o **primeiro pacote UDP vindo de h2**
- ou **pacotes pequenos (< 100 bytes)**, já que handshake é pequeno
- ou **primeiro fluxo visto**

Exemplo **por tamanho** (match restrito ao tamanho do payload IP):

```
match = parser.OFPMatch(  
    eth_type=0x0800,  
    ip_proto=17,  
    udp_dst=4433,  
    ip_total_length=(<valores pequenos>  
)
```

Mas esse campo nem sempre disponível.

Então o método recomendado é:

👉 **ver a primeira PACKET_IN do fluxo e dropá-la:**

```
if is_first_time_seen(flow_key):  
    actions = []  
    self.add_flow(datapath, 400, match, actions)
```

onde `flow_key = (eth_src, eth_dst, udp_dst)`.

7) Priorizar QUIC-sim

Criar um flow com prioridade alta:

```
match = parser.OFPMatch(eth_type=0x0800, ip_proto=17, udp_dst=4433)
actions = [parser.OFPActionOutput(PORTO_PREFERENCIAL)]

self.add_flow(datapath, priority=300, match=match, actions=actions)
```

Quando existir conflito, o switch escolhe a regra com maior prioridade.

8) Rerroteamento sob falha de link

Você simula falha no Mininet:

```
mininet> link s1 s2 down
```

No controlador, ao receber PACKET_IN indicando que pacote não pode ser entregue pela porta anterior:

```
if dpid == 1:
    # Porta primária (caminho superior) está indisponível?
    if not self.port_live(dpid, PORTO_PARA_S2):
        print("[SDN] Falha detectada. Redirecionando QUIC-sim para rota inferior.")
        match = parser.OFPMatch(eth_type=0x0800, ip_proto=17, udp_dst=4433)
        actions = [parser.OFPActionOutput(PORTO_PARA_S3)]
        self.add_flow(datapath, 300, match, actions)
```

A função `port_live()` precisa ser implementada ou substituída pela lógica:

- se PACKET_IN vem sempre da porta errada
- se o destinatário some da tabela ARP
- etc.

Ou simplesmente:

Se caminho superior não está previamente instalado → instale inferior.

9) Redirecionamento baseado no MAC do host

Se você quiser que tráfego de h1 siga um caminho e h2 outro:

```
if eth.src == "00:00:00:00:00:01":
    out_port = PORTO_PARA_S2
else:
    out_port = PORTO_PARA_S3

actions = [parser.OFPACTIONOutput(out_port)]
self.add_flow(datapath, 200, match, actions)
```

10) Manipulação baseada em porta fonte (ataques, DoS, filtros)

Exemplo: bloquear QUIC-sim vindo de uma porta específica:

```
match = parser.OFPMATCH(
    eth_type=0x0800,
    ip_proto=17,
    udp_src=4433,
    udp_dst=4433
)

actions = []
self.add_flow(datapath, 400, match, actions)
```



Resumo: Principais padrões de regras SDN

Ação	Implementação
Forçar caminho superior	<code>Output(PORTO_PARA_S2)</code>
Forçar caminho inferior	<code>Output(PORTO_PARA_S3)</code>
Balanceamento ECMP-sim	<code>Output(random.choice(...))</code>
Bloqueio (DROP)	<code>actions = []</code>
Priorizar QUIC-sim	Prioridade alta (200–500)
Desviar fluxo em falha	Detectar e trocar porta de saída
Redirecionar por host	Match baseado em <code>eth.src</code>
Redirecionar por porta	Match baseado em <code>udp_src</code> ou <code>udp_dst</code>